

MODÈLE POUR SYNTHÈSE PARAMÉTRIQUE (VITAL + MAX4LIVE + IA)

Benard Amaury

Objectif général

Développer un modèle IA capable de retrouver les paramètres d'un synthétiseur (Vital) à partir d'un audio donné, de façon à reconstituer automatiquement un preset.

Le dataset sera construit à l'aide d'un device Max for Live générant des paramètres aléatoires en sauvegardant simultanément :

l'audio généré,

la liste des paramètres exacts utilisés.

PLAN DE PROJET

1) CHECKPOINT 1 — 19 Décembre

Objectif principal : pipeline complet pour la génération du dataset + premiers modèles simples.

PARTIE A — Génération automatisée du dataset

A1. Mise à jour du device Max for Live pour logging automatique

Description :

Le device doit écrire dans un fichier texte ou JSON, pour chaque preset généré :

- ID unique
- liste complète des paramètres envoyés à Vital

Implémentation en plusieurs pistes possibles :

Piste 1 : JavaScript dans Max

Utilisation de `dict + dict.serialize` pour écrire du JSON.

Piste 2 : coll

Sauvegarde dans un fichier texte `.txt` via `write`.

Piste 3 : Node for Max

Écriture directe d'un fichier JSON via `Node.js`.

optionnel :

Ajouter des noms de paramètres lisibles.

Ajouter une interface graphique de monitoring.

A2. Automatisation de la génération audio

Description :

L'objectif est de générer plusieurs centaines de sons :

un paramètre aléatoire → un preset → export audio 5 sec.

Pistes techniques :

Automatisation Ableton : lancer un enregistrement continu, découper ensuite par timestamp.

Ou utiliser Max pour déclencher automatiquement l'export d'un clip.

Ou enregistrer un flux audio brut et découper offline via Python (librosa).

Optionnel :

Ajouter un système réduisant les presets absurdes (exemple : volumes trop faibles).

A3. Structuration finale du dataset

Description :

Créer une structure claire :

```
dataset/  
  audio/  
    preset_0001.wav  
    preset_0002.wav  
    ...  
  params.json
```

où params.json contient un dictionnaire {id_preset: {paramètres ...}}.

Optionnel :

Convertir les audios en spectrogrammes (librosa STFT ou mel spectrogram).

Normaliser tous les paramètres dans [0,1] immédiatement.

PARTIE B — Premiers modèles de reconstruction de paramètres

B1. Prétraitement audio → feature vector

Description :

Pour prédire correctement les paramètres, l'audio d'entrée doit être transformé en représentation stable :

Pistes possibles :

Spectrogramme Mel (librosa.melspectrogram) — le plus simple et robuste.

MFCC.

STFT magnitude seule.

Optionnel :

Ajouter une compression logarithmique.

Tester plusieurs tailles de fenêtre.

B2. Modèle simple de régression

Description :

Architecture initiale :

CNN2D ou CNN1D → flatten → MLP → paramètres Vital prédits.

Objectif :

Vérifier que les paramètres les plus simples (ex : oscillateur principal, filtre cutoff, enveloppe amplitude) sont prédictibles.

Optionnel :

Tester des architectures plus lourdes : CRNN, transformer, autoencoder audio.

Ajouter un modèle résiduel pour affiner la reconstruction.

B3. Évaluation du modèle et validation

Description :

Comparer :

paramètres réels vs prédictions (MSE)

écart sonore entre audio réel et audio re-synthétisé avec les paramètres prédits (optionnel).

Optionnel :

Utiliser mir_eval pour un score de similarité audio.

Objectif final du Checkpoint 1 (19/12)

Device Max4Live mis à jour et sauvegardant les paramètres.

Pipeline complet permettant de générer automatiquement dataset audio + paramètres.

Prétraitement audio fonctionnel.

Premier modèle prédictif entraîné, même simple. (si j'ai le temps)

2) CHECKPOINT 2 — 2 Janvier

Objectif principal : modèle prédictif stable + début de l'intégration Max4Live → Python → Vital.

PARTIE A — Amélioration de la qualité du modèle

A1. Sélection d'un sous-ensemble de paramètres pertinents

Description :

Vital possède énormément de paramètres.

Priorité :

Oscillator 1 (waveform, wavetable pos, unison, level)

Filter 1 (cutoff, resonance)

Envelope 1 (A, D, S, R)

FX BYPASS → optionnel

LFO → optionnel

Optionnel (si temps disponible) :

Ajouter les effets (delay, reverb, distortion, chorus).

Ajouter les modulations complexes (LFO2, LFO3, random).

A2. Enrichissement du dataset

Description :

Générer davantage de presets pour augmenter la diversité sonore.

Indispensable pour améliorer la généralisation.

Optionnel :

Stratifier les paramètres (ex : favoriser des cutoffs extrêmes).

A3. Entraînement d'un modèle plus avancé

Pistes techniques :

CRNN (CNN + GRU) — très efficace pour l'analyse audio.

Transformer encoder appliqué au spectrogramme.

Model multi-head avec sortie structurée par catégories de paramètres.

Optionnel :

Ajouter un autoencodeur audio pour régulariser l'apprentissage.

Utiliser un VAE pour relier latent → paramètres.

PARTIE B — Début de l'intégration dans Max for Live

B1. Communication Max <-> Python

Pistes techniques :

OSC via udp send et udp receive.

Node for Max appelant un script Python.

maxpy (API Python officielle Max).

Objectif :

Envoyer un audio (ou spectrogramme) à Python.

Recevoir les paramètres prédits et les renvoyer à Vital.

B2. Recréation automatique d'un preset dans Vital

Description :

Python → Max → Vital (via automation)

Vital doit recevoir :

cutoff

resonance

envelope

oscillateur 1 shape

etc.

Optionnel :

Ajouter la possibilité d'enregistrer directement un preset .vital.

Objectif final du Checkpoint 2 (2/1)

Modèle prédictif stable sur les paramètres principaux.

Dataset enrichi.

Communication Python ↔ Max opérationnelle.

Reconstruction automatique de presets simples dans Vital via IA.

3) CHECKPOINT 3 — 15 Janvier

Objectif principal : système complet attentif à l'utilisateur + refinements + résultats présentables.

PARTIE A — Stabilisation et intégration finale

A1. Optimisation du modèle

Réduction de l'erreur.

Meilleure normalisation des paramètres.

Ajustement des architectures.

Optionnel :

Fine-tuning par catégorie sonore (bass, pad, lead).

A2. Interface Max for Live finale

Description :

Interface simple avec :

bouton "Analyze Sound"

visualisation du spectrogramme (optionnel)

affichage des paramètres reconstruits

bouton "Send to Vital"

Optionnel :

Afficher la confiance du modèle.

Ajouter un mode "preset morphing".

PARTIE B — Tests d'usage réalistes

Description :

Tester l'analyse d'un son In The Wild.

Vérifier la cohérence des paramètres retrouvés.

Comparer audio original vs audio re-synthétisé.

Optionnel :

Ajouter une étape d'optimisation locale par gradient ou par search autour des paramètres prédits.

Objectif final du Checkpoint 3 (15/1)

Modèle capable d'inférer les paramètres principaux de Vital à partir d'un audio donné.

Device Max4Live complet : analyse → reconstruction automatique.

Résumé final

Génération automatique dataset Max4Live

Logging des paramètres dans un fichier JSON

Prétraitement audio (spectrogramme)

Régression audio → paramètres

Reconstruction automatique du preset

Communication Max ↔ Python

Reconstruction dans Vital

Optionnel (si temps)

Gestion des effets et filtres avancés

Modulations complexes (LFO multiples, matrice de modulation)

Optimisation locale post-prédiction

Interface utilisateur enrichie

Analyse du son en temps réel

Modèle basé sur VAE ou transformer très large

Reconstruction complète d'un preset Vital .vital